ISTA 130

`while` Loops

Nested Conditionals

- The body of a conditional statement can of course contain other conditional statements
  - We call this *nesting*

```
if condition:
    ...
    if condition:
        ...
    else:
        ...
    ...
elif condition:
    ...
    if condition:
        ...
    ...
else:
    ...
```

Short-Circuit Evaluation

```
if y != 0 and abs(x / y) < 1:
    print('safe to divide')

if cheap_comp(a, b) or expensive(a, b):
    print('saved computational expense')
```

Non-Boolean Values as Conditions

- Try this:

```
name = 'Snoopy'
if name:  # wait, what???
   print('Favorite cartoon dog.')
```

- Python considers any value that is "non-zero" to be `True`
- Try changing the first line to: `name = ''`
  - Python considers the "empty string" (`''`) to be `False`
- Since Python has a "loose" interpretation of Boolean values, it is easy to make some kinds of mistakes.
  - What we probably wanted above was a comparison:

```
name = 'Snoopy'
if name == 'Snoopy':
   print('Favorite cartoon dog.')
```

- It is best/safest to strictly use expressions that evaluate to either `True` or `False` in your conditions.

Boolean Expressions

- This won't do what you want:

```
a = 12
b = 10
c = 0

if a > b and c:
   print('yup')
```

- Instead do it like this:

```
if a > b and a > c:
   print('yup')
```

Values as Conditions in Python

- Zero is `False`.
- Empty collections are `False`.
- All other values are `True`.

Type	False	True
`int`	`0`	All other values
`float`	`0.0`	All other values
`str`	`'', ""`	All other values
`list`	`[]`	All other values
`dict`	`{}`	All other values

Operators in Conditions

Kind of Operator	Operators
Binding	`()`
Arithmetic	`**, *, /, //, %, +, -`
Comparison	`==, !=, <, >, <=, >=`
Membership	`in, not in`
Logical	`not, and, or`

Conditions

Assume the function `is_adult(age)` takes an int parameter and returns a boolean value.

```
legal = is_adult(age)
if legal == True:
   print('Have a beer')
```

- Is the comparison to `True` necessary?
  - `Python` always *evaluates* the condition, boiling it down to either `True` or `False`
  - The comparison to `True` is redundant
  - It's like asking "Is True True?"

Conditions

- Instead just write this:

```
legal = is_adult(age)
if legal:
   print('Have a beer')
```

- Note that we wouldn't do this:

```
if (height > 60) == True:
   print('quite tall')
```

- and we certainly wouldn't do this:

```
if ((legal == True) == True) == True:
   print('Have a beer')
```

Most of the time, concise == clear.

Loops

- The `for` loop is used when we know beforehand how many *iterations* we need (i.e. how many times we need to go through the loop).
  - the number might be fixed (e.g. drawing a square)
  - the number might come from user input
- Sometimes we don't know beforehand
  - e.g. a loop that repeats until the user decides to quit
  - `for i in range(???)`... the number of iterations is unknown
- The `while` loop allows us to repeat code in cases where we don't know.

`while` Loop

The `while` loop looks similar to a conditional statement:

```
if condition:
    statement(s) to execute

while condition:
    statement(s) to repeat
```

- The difference is that the **while** loop jumps back up to *check the condition again* each time it finishes executing the body
  - (i.e. while the condition is True, keep repeating the body)

`while` Loop vs. `for` Loop

- A simple for loop:

```
for i in range(10):
    print(i)
```

- The equivalent while loop:

```
i = 0
while i < 10:
    print(i)
    i = i + 1
```

`while` Loop

- Like the `for` loop, the `while` loop executes the code in its body (the indented code) and then repeats.

```
i = 0
while i < 10:
    print(i)
    i = i + 1
```

- Reading this out loud:
  - `i` gets assigned 0
  - while `i` is less than 10 repeat the following:
    - print the value that `i` refers to
    - increase the value `i` refers to by 1

- Let's keep a written trace as we walk through this

`while` Loop vs. `for` Loop: Loop Control Variable

- for loop:

```
for i in range(10):
    print(i)
print('After loop:', i)
```

- equivalent while loop:

```
i = 0
while i < 10:
    print(i)
    i = i + 1
print('After loop:', i)
```

`for` vs. `while`: Loop Control Variable (LCV)

	`for` loop	`while` loop
Who initializes LCV?	Python	You
Who changes LCV?	Python	You
Who checks LCV to see if done?	Python	You

Burp

Printing burp:

```
word = ''
i = 0
while i < 10:
    print(i, ' ', word)
    i = i + 1
    word = 'burp' * i

print('Done:')
print(i, ' ', word)
```

Broken Burp

Breaking the burp printer:

- What will this do?

```
word = ''
i = 0
while i > 10:
    print(i, ' ', word)
    i = i + 1
    word = 'burp' * i

print('Done:')
print('i =', i, ', word =', word)
```



What Else Can Go Wrong?



Really Broken Burp!

```
word = ''
i = 0
while i >= 0:
    print(i, ' ', word)
    i = i + 1
    word = 'burp' * i

print('Done:')
print(i, ' ', word)
```

- My favorite error- **infinite** loop!
  - The condition is re-evaluated each time the body finishes
    but this condition will always be `True`... forever!
- Don't panic
  - just type `ctrl-c` in your shell
  - or close the terminal window

Another Burp Mistake

```
word = 'burp'
i = 0
while i < 10:
    print(i, ' ', word)
    word = 'burp' * i

print('Done:')
print(i, ' ', word)
```

- This is another **infinite** loop
  - The condition is re-evaluated each time the body finishes
  - Since the condition never changes it will always be `True`... forever!
  - (remember: `ctrl-c` will kill the program)

`while` Loop

So, when writing a `while` loop you need to do several things:

1. Choose the "entry" condition
   - what must be `True` to execute the body?
   - like an `if`, the body won't be entered if the condition is `False`
2. Add the body
   - i.e. the statements to be repeated (indent them)
3. Make sure that the code in the body will eventually cause the condition to become `False`!
   - usually by changing (e.g. incrementing or decrementing) the value of a variable used in the condition

How Many Times will this Loop Iterate?

```
1     word = 'nose'
2     wrong = True
3
4     while wrong:
5         guess = input('Guess the word: ')
6         if guess == word:
7             wrong = False
8         else:
9             print('Wrong!')
10
11    print('GOT IT!')
```

Clearer?

```
1     word = 'nose'
2     guessed_it = False
3
4     while not guessed_it:
5         guess = input('Guess the word: ')
6         if guess == word:
7             guessed_it = True
8         else:
9             print('Wrong!')
10
11    print('GOT IT!')
```

`while` Loop in a Function

```
1  def guess_word(word):
2      guessed_it = False
3      count = 0
4      while not guessed_it:
5          guess = input('Guess the word: ')
6          count = count + 1
7          if guess == word:
8              guessed_it = True
9          else:
10             print('Wrong!')
11     return count
12
13 #=============================
14 def main():
15     word = 'bugs'
16     guesses = guess_word(word)
17
18     print('You took', guesses, 'guesses')
```